

Lab 3) Wall following.

In this lab, I used the provided python skeleton code to solve the left wall following problem. Initially, I refined the getRange function such that based on the provided angle, it finds the index associated with that angle in the matrix data from Lidar and returns the distance at that angle. Next, in the follow left function, I calculate the a and b parameters which are crucial to calculating steering angle and error. Finally, with the error returned from the follow left function, the PID control function calculates the desired steering angle. In tuning PID controller gains, I started with tuning Kp and after reaching to my desired values based on experiments, I tuned Ki and Kd to smooth the car motion.

One of the significant challenges I encountered during this exercise was identifying the correct angles corresponding to the vehicle's left side. Additionally, determining an optimal value for the theta angle was a little challenging. To further my understanding and to compare performance metrics, I developed the program in both Python and C++. Despite both versions embodying the same logical structure and sharing identical PID gains, the performance difference was notable. The C++ version exhibited superior functionality, prompting me to favor it for this discussion.

Code:

```
#include <ros/ros.h>
#include <sensor_msgs/LaserScan.h>
#include <ackermann_msgs/AckermannDriveStamped.h>
#include <cmath>
#include <algorithm>

// PID CONTROL PARAMS
double kp = 0.55;
double kd = 0.2;
double ki = 0.0001;
double servo_offset = 0.0;
double prev_error = 0.0;
double error = 0.0;
double integral = 0.0;

// WALL FOLLOW PARAMS
const int ANGLE_RANGE = 360;
const double DESIRED_DISTANCE_RIGHT = 0.9;
const double DESIRED_DISTANCE_LEFT = 0.55;
const double VELOCITY = 2;
const double CAR_LENGTH = 0.5;

class WallFollow
{
public:
    WallFollow()
    {
        // Topics & Subs, Pubs
        std::string lidarscan_topic = "/scan";
        std::string drive_topic = "/drive";
        lidar_sub = nh.subscribe(lidarscan_topic, 1000, &WallFollow::lidar_callback, this);
        drive_pub = nh.advertise<ackermann_msgs::AckermannDriveStamped>(drive_topic, 10);
    }

    void lidar_callback(const sensor_msgs::LaserScan::ConstPtr& data)
    {
        double error = followLeft(data, DESIRED_DISTANCE_LEFT);
        pid_control(error, VELOCITY);
    }
}
```

```

private:
    ros::NodeHandle nh;
    ros::Publisher drive_pub;
    ros::Subscriber lidar_sub;

    double getRange(const sensor_msgs::LaserScan::ConstPtr& data, int angle)
    {
        int index = angle * 3;
        double dist = data->ranges[index];
        if (std::isnan(dist) || std::isinf(dist)) {
            return 0.0;
        } else {
            return dist;
        }
    }

    void pid_control(double error, double velocity)
    {
        double angle = 0.0;
        integral += error;
        double derivative = error - prev_error;
        angle = -(kp * error + ki * integral + kd * derivative);
        prev_error = error;
        ROS_INFO("Angle: %f", angle * 180.0 / M_PI);

        if (std::abs(angle) <= 10.0 * M_PI / 180.0) {
            velocity = 1.5;
        } else if (std::abs(angle) <= 20.0 * M_PI / 180.0) {
            velocity = 1.0;
        } else {
            velocity = 0.5;
        }

        ackermann_msgs::AckermannDriveStamped drive_msg;
        drive_msg.header.stamp = ros::Time::now();
        drive_msg.header.frame_id = "laser";
        drive_msg.drive.steering_angle = angle;
        drive_msg.drive.speed = velocity;
        drive_pub.publish(drive_msg);
    }

    double followLeft(const sensor_msgs::LaserScan::ConstPtr& data, double leftDist)
    {
        // Follow left wall as per the algorithm
        double a = getRange(data, 210);
        double b = getRange(data, 270);
        double alpha = atan((a * cos(M_PI / 3) - b) / (a * sin(M_PI / 3)));
        ROS_INFO("Alpha: %f", alpha);
        double D_t = b * cos(alpha);
        double D_tplus1 = D_t + sin(alpha) * CAR_LENGTH;
        double error = (DESIRED_DISTANCE_LEFT - D_tplus1);
        ROS_INFO("Dt: %f", D_tplus1);
        ROS_INFO("Error: %f", error);
        return error;
    }
};

int main(int argc, char** argv)
{
    ros::init(argc, argv, "WallFollow_node");
    WallFollow wf;
    ros::spin();
    return 0;
}

```